
Parallel and Distributed Computing

Maurice Oliver Y. Dela Cruz

Chapter 6: Parallel Program Development



Chapter 6: Parallel Program Development

Objective:

Apply parallel programming concepts to real-world problems.

Key Concepts

Developing
parallel n-body
solver algorithms.

Balancing
workload and
optimizing
communication in
hybrid systems.

Comparing the
performance of
MPI, Pthreads,
and OpenMP
implementations.

What is the N-Body Problem?

- Simulates motion of particles under forces (e.g., gravity)
- Each body interacts with every other body

What is the N-Body Problem?

Formula:

$$F = \frac{Gm_1m_2}{r^2}$$

F - gravitational force between two bodies

G - gravitational constant

m_1, m_2 - masses of the two bodies

r - distance between them

Computational Challenge

- Each body interacts with others
- Time complexity:

$$O(n^2)$$

Becomes very slow for large n

Why Parallelize?

- Reduce execution time
- Handle large simulations
- Utilize
 - Multi-core CPUs
 - Clusters

MPI Overview

- MPI - Message Passing Interface
- Used for distributed memory systems
- Each process has its own memory

– MPI Program Structure

1. Initialize MPI
2. Divide data among processes
3. Communicate data
4. Compute forces
5. Update positions

– Data Distribution

Total bodies split across the process

$$\text{local_n} = \frac{\text{total bodies}}{\text{processes}}$$

Each process handles a subset

– Communication Step

```
MPI_Allgather(...)
```

Purpose:

- Share all body data across processes
- Each process gets complete system state

– MPI Force Computation

- Each process computes force for its local process
- Uses global data

– MPI Strengths and Weaknesses

- Scales across multiple machines
- Communication overhead is high

OpenMP Overview

- Shared memory model
- Uses compiler directives
- Runs on multi-core CPUs

Parallel Loop Concept

```
#pragma omp parallel for
```

Purpose:

- Splits iterations across threads

Force Computation

- Outer loop parallelized
- Each thread computes force independently

Position Updates

- Also parallelized using OpenMP
- No communication required

– OpenMP Strengths and Weaknesses

- Easy to implement
- Fast on single machine
- Limited scalability (no cluster support)

– Pthreads Overview

- Low-level threading library
- Programmer manually controls threads

Thread Work Division

- Divide bodies into chunks
- Each thread processes a portion

Thread Function

Each thread

- Compute forces
- Updates positions

Thread Life Cycle

```
pthread_create(...)
```

```
pthread_join(...)
```

- Create threads
- Wait for completion

– Pthreads Strengths and Weaknesses

- Full control of threads
- Complex and error-prone

Comparison Table

Feature	MPI	OpenMP	Pthreads
Memory	Distributed	Shared	Shared
Ease	Medium	Easy	Hard
Scale	High	Medium	Medium

– Key Insight

All three approaches:

- Solve the same physics problem
- Differ on how work is divided

Performance Considerations

- MPI - communication cost
- OpenMP - thread overhead
- Pthreads - manual scheduling

Real-World Improvements

- Barnes-Hut Algorithm - $O(n \log n)$
- GPU Acceleration
- Hybrid (MPI + OpenMP)

Summary

- N-body problem is computational intensive
- Parallelism significantly improves performance
- Choice depends on:
 - Hardware
 - Problem size

Discussion Questions

- Why MPI is slower in small problems?
- When you should use OpenMP over Pthreads?

—

Thank you for listening.

